

Kernel Image Processing

Sofija Kochovska

89221062@student.upr.si

FAMNIT, University of Primorska

Koper, Slovenia

ABSTRACT

This report explores the application of kernel-based image processing techniques for modifying digital images. It provides an overview of the convolution process using kernels and presents a comparative study of three implementation approaches: sequential, parallel, and distributed. Each method is implemented and evaluated based on its performance and efficiency. The results highlight the advantages and trade-offs of each approach, ultimately identifying the most effective solution for processing high-resolution images using kernel operations.

KEYWORDS

kernel image processing, convolution, sequential processing, parallel computing, distributed computing, performance evaluation

1 INTRODUCTION

Image processing plays a vital role in many modern applications, from photography and computer vision to medical imaging and machine learning. A fundamental technique used in image processing is the application of kernels, which are small matrices of numbers designed to alter pixel values in a controlled way. By convolving a kernel across an image, it is possible to apply various effects such as blurring, sharpening, edge detection, and embossing. These effects are not only useful for visual enhancement but are also critical in feature extraction, which helps identify key patterns or structures within images [2].

This project explores the implementation of kernel-based image processing using three different approaches: sequential, parallel, and distributed computing. Each approach processes the image using the same core convolution logic, but leverages different system resources to handle computational tasks. The aim is to compare their performance and evaluate which method offers the most efficient solution for processing large images with kernel filters.

2 CONVOLUTION IN IMAGE PROCESSING

In image processing, a *kernel*, also called a convolution matrix or mask, is a small matrix used for blurring, sharpening, embossing, edge detection, and other effects. This is accomplished by performing a convolution between the kernel and the image. More simply, each pixel in the output image is computed as a weighted sum of the nearby pixels (including itself) in the input image, where the weights are given by the kernel. The kernel is moved across the entire image to apply this operation to every pixel, creating the filtered output.

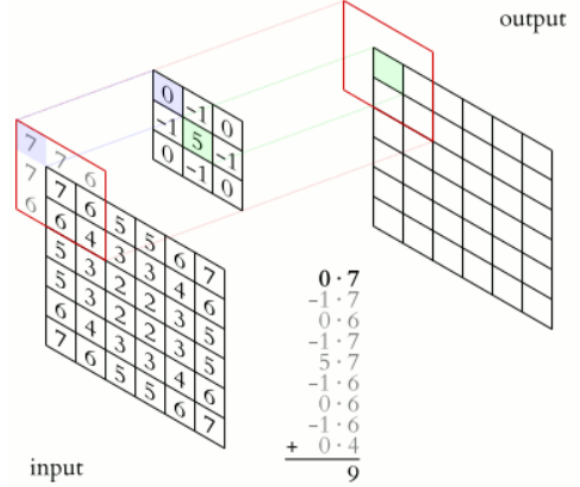


Figure 1: Convolution of an image with a kernel, illustrating how the output pixel is calculated. [1]

2.1 Definition of Convolution

The convolution operation is defined as:

$$g_{x,y} = \omega * f_{x,y} = \sum_{i=-a}^a \sum_{j=-b}^b \omega_{i,j} f_{x-i,y-j} \quad (1)$$

where:

- $g(x, y)$ is the filtered (output) image,
- $f(x, y)$ is the original (input) image,
- ω is the kernel (filter) of size $(2a + 1) \times (2b + 1)$,
- i and j index the kernel elements over the range $-a \leq i \leq a$, $-b \leq j \leq b$.

2.2 Common Kernels and Their Effects

Depending on the kernel values, different image processing operations can be performed:

Identity kernel [3]. Leaves the image unchanged:

$$\omega = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Ridge or edge detection kernels [3]. Detect edges by highlighting regions with rapid intensity changes:

$$\omega_1 = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad \omega_2 = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Sharpen kernel [3]. Enhances edges and fine details:

$$\omega = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

3 IMPLEMENTATION

This project was implemented in Java using the AWT and Swing libraries to handle image input/output and display. The core logic is structured around a shared interface and modular classes that allow for sequential, parallel, and distributed implementations of kernel-based image processing.

3.1 Design Overview

A common interface, `IMGProcessor`, was created to ensure consistency across all implementations. It defines the essential methods for grayscale conversion, kernel convolution, and full image processing:

```
public interface IMGProcessor {
    BufferedImage convertToGrayscale(BufferedImage colorImage);
    BufferedImage applyConvolution(BufferedImage inputImage,
        int[][] kernel);
    BufferedImage processImage(String inputFilePath, int[][]
        kernel);
}
```

Each implementation class (sequential, parallel, and distributed) implements this interface.

3.2 Sequential Implementation

The sequential implementation is defined in the `ImgProcMethods` class, which performs all operations on a single thread. The main steps include:

- **Grayscale Conversion:** The `convertToGrayscale` method processes each pixel in the image using the standard luminance formula to compute a grayscale value. A new grayscale image is returned.
- **Kernel Convolution:** The `applyConvolution` method slides the given kernel across the grayscale image and computes the filtered pixel values. Edge handling is done by skipping out-of-bounds coordinates. The results are clamped to ensure valid pixel intensities.
- **Image Processing Pipeline:** The `processImage` method ties everything together by loading the image from disk, converting it to grayscale, and applying the convolution kernel.

The execution begins from the `SMain` class, which creates an instance of the sequential processor and launches the graphical user interface (GUI) for user interaction.

The pseudocode for this implementation is summarized in Algorithm 1.

Algorithm 1 Sequential Implementation - `ImgProcMethods.java`

```
1: Class ImgProcMethods implements IMGProcessor
2: function CONVERTTOGRAYSCALE(colorImage)
3:   For each pixel in colorImage
4:     Compute weighted average of RGB values
5:     Set grayscale pixel in output image
6:   Return grayscale image
7: end function
8: function APPLYCONVOLUTION(grayImage, kernel)
9:   For each pixel ( $x, y$ ) in grayImage
10:    Compute weighted sum using kernel
11:    Clamp result to [0, 255]
12:    Store result in output image
13:   Return filtered image
14: end function
15: function PROCESSIMAGE(inputPath, kernel)
16:   Load input image from file
17:   Convert to grayscale
18:   Apply convolution kernel
19:   Return final image
20: end function
```

3.3 Parallel Implementation

For the parallel implementation, I used Java's `ForkJoinPool` to leverage all available CPU cores. The idea is to process each pixel independently and in parallel, which significantly reduces processing time for large images.

In the `convertToGrayscale` method, I flatten the 2D image pixels into a 1D stream and submit them to the `ForkJoinPool`. Each thread takes pixels to convert from color to grayscale using a weighted average formula. This parallel approach efficiently distributes the workload without explicit thread management.

Similarly, in the `applyConvolution` method, the convolution operation is parallelized. Each pixel's neighborhood is processed in parallel to compute the filtered output. Boundary checks ensure pixels outside the image area are ignored.

I chose `ForkJoinPool` for several reasons:

- It dynamically balances workload with a work-stealing algorithm, improving CPU utilization.
- It reuses threads instead of creating new ones, minimizing overhead.
- It is part of the standard Java library, making it easy to integrate without external dependencies.

The pseudocode for the parallel processing is shown in Algorithm 2. By splitting the image processing task into many small independent units (pixels), the program can fully utilize multi-core processors, resulting in significant performance gains compared to sequential processing. Additionally, this approach scales well with the number of available CPU cores, making it adaptable to different hardware configurations without changing the code. Using parallel streams combined with `ForkJoinPool` abstracts away much of the thread management complexity, allowing me to write clean and maintainable code while achieving high efficiency. This parallel design ensures that the image processing operations remain responsive even when working with high-resolution images.

Algorithm 2 Parallel Implementation in PImgProcMethods.java

```

1: Create a ForkJoinPool with threads = number of processors
2: Submit a parallel task to the pool:
3: for all pixel index  $i \in [0, \text{width} \times \text{height})$  in parallel do
4:    $x \leftarrow i \bmod \text{width}$ 
5:    $y \leftarrow i \div \text{width}$ 
6:   if applying grayscale conversion then
7:     Extract RGB values at  $(x, y)$ 
8:     Compute grayscale value using weighted formula
9:     Set grayscale pixel in output image at  $(x, y)$ 
10:  else if applying convolution then
11:    Initialize accumulator to 0
12:    for all kernel element  $(k_x, k_y)$  do
13:      Calculate neighbor pixel coords  $(p_x, p_y) \leftarrow (x +$ 
 $k_x - \text{halfKernel}, y + k_y - \text{halfKernel})$ 
14:      if  $(p_x, p_y)$  inside image bounds then
15:        Multiply pixel value by kernel coefficient and
        add to accumulator
16:      end if
17:    end for
18:    Clamp accumulator to  $[0, 255]$ 
19:    Set output pixel color at  $(x, y)$ 
20:  end if
21: end for
22: Wait for all tasks to complete (join)

```

4 DISTRIBUTED IMPLEMENTATION

For distributed computing, I used MPI (Message Passing Interface) through the MPJ Express library. The image processing task was divided by splitting the input image into smaller horizontal chunks, which were distributed to worker processes for parallel processing. Each worker applied the convolution kernel on its assigned chunk and sent the processed results back to the master process. The master then combined the processed chunks, carefully handling overlapping border regions caused by the convolution kernel, to form the final processed image.

This approach leverages parallelism to accelerate the computationally intensive convolution step, reduces memory overhead by working directly with raw grayscale pixel byte arrays instead of full BufferedImage objects, and minimizes communication costs by sending only essential raw data between processes.

4.1 Key Methods

- **processImage (master):** This method runs only on the master process. It reads the original image, converts it to a grayscale byte array, and partitions the image into chunks based on the number of worker processes and kernel size. To handle convolution border effects, it adds overlapping rows between chunks equal to half the kernel height. The master sends each chunk along with the convolution kernel to each worker. After processing, it receives the processed chunks and stitches them back into the final image, discarding overlapping borders to avoid duplication.
- **workerProcess (worker):** The worker processes run this method, which enters a loop waiting for commands from

the master. Upon receiving chunk dimensions, pixel data, and kernel data, the worker reconstructs the kernel matrix and applies the convolution on the received grayscale chunk using an efficient byte-array-based convolution method. The processed result is sent back to the master. The worker continues this loop until it receives a stop signal.

- **applyConvolution (shared):** This method performs the core convolution operation on raw byte arrays representing grayscale image data. Operating directly on byte arrays avoids the overhead of repeatedly creating BufferedImage objects and allows memory-efficient reuse of buffers. The method carefully handles image boundaries by skipping pixels where the kernel would overflow.

4.2 Design Decisions and Rationale

- **Use of Raw Byte Arrays:** Instead of repeatedly converting between BufferedImage and grayscale images, the implementation works directly with raw byte arrays for pixel data. This reduces memory usage and speeds up processing, which is especially beneficial when distributing large images over MPI.
- **Chunk Partitioning with Overlaps:** To correctly apply convolution near chunk borders, overlapping rows equal to half the kernel height are included in each chunk except for the first and last. This prevents edge artifacts and ensures seamless stitching without artifacts or data loss.
- **Flattened Kernel Transmission:** The convolution kernel, represented as a 2D integer array, is flattened into a 1D integer array before MPI transmission to simplify sending and receiving via MPI, which works best with primitive arrays.
- **Master-Worker Communication Protocol:** The master sends the image chunk dimensions and pixel data, followed by kernel dimensions and kernel data, to each worker. Workers send back the processed chunk dimensions and results. This structured communication ensures synchronization and correct chunk identification when reassembling the final image.
- **Stop Signal Implementation:** A dedicated MPI tag and zero-length message are used to signal workers to terminate gracefully after processing completes. This clean shutdown prevents deadlocks and resource leaks in the distributed system.
- **Single Master GUI Execution:** The GUI runs only on the master process to avoid conflicts. Workers focus solely on computation, enabling a clear separation of concerns and simplifying the user interface logic.

4.3 Main Entry Point

The main entry point of the distributed application is defined in the DMain class. It initializes the MPI environment and distinguishes between the master process and worker processes based on the MPI rank.

- **MPI Initialization:** The program starts by calling `MPI.Init(args)` to initialize the MPI environment, enabling inter-process communication.

- **Rank and Size Determination:** Each process retrieves its unique rank and the total number of processes `size` from the MPI communicator `COMM_WORLD`. The rank determines the role of the process (master if 0, worker otherwise).
- **Minimum Process Check:** The program checks if at least two MPI processes are available (one master and at least one worker). If not, the master outputs an error and the program exits cleanly.
- **Master Process Responsibilities:**
 - The master process (`rank == 0`) creates an instance of the image processing methods handler `DImgProcMethods`.
 - It launches the graphical user interface (GUI) using `GUI.run(processor)`, allowing the user to interactively control the image processing task.
 - After the GUI closes (blocking call), the master sends stop signals to all worker processes using `processor.stopWorkers(size - 1)` to gracefully terminate their execution.
- **Worker Process Responsibilities:**
 - Each worker process (`rank != 0`) creates its own instance of `DImgProcMethods`.
 - It enters the `workerProcess(rank)` method, which contains a loop that waits for image chunk data and kernel information from the master, processes assigned chunks, and sends results back until a stop signal is received.
- **MPI Finalization:** Finally, all processes call `MPI.Finalize()` to clean up MPI resources and end the parallel program.

4.4 Design Rationale of Main

- **Role Assignment via MPI Rank:** Using MPI ranks to differentiate the master and workers is a standard and effective way to distribute roles in MPI programs, simplifying the coordination logic.
- **GUI on Master Only:** Running the GUI exclusively on the master prevents concurrency and synchronization issues related to multiple GUIs across processes, concentrating user interaction on a single interface.
- **Graceful Worker Shutdown:** The explicit stop signal ensures workers do not continue waiting indefinitely, preventing deadlocks and allowing orderly program termination.
- **Separate Instances for Master and Workers:** Creating separate instances of the processing methods class for master and workers allows the encapsulation of master-specific and worker-specific logic in the same class while keeping process roles distinct.
- **Blocking GUI Call:** Running the GUI in a blocking manner on the master ensures that worker processes remain active during user interaction, and processing commands are only sent when the user triggers them.

4.5 Performance Considerations

While distributed computing can theoretically provide significant performance improvements, the actual gains depend on several factors:

- **Network Latency:** The time taken to send and receive chunks over the network can impact overall performance. High latency slows down communication between the master and workers, potentially causing idle time.
- **Data Transfer Overhead:** Large images involve substantial data transfer between processes. This overhead can diminish the benefits of parallel processing if communication costs outweigh computation time.
- **Synchronization:** Combining processed chunks correctly requires synchronization among all workers and the master. This coordination can introduce delays, especially if some workers finish much earlier than others (load imbalance).

5 TESTING ENVIRONMENT

I tested the program on my laptop computer with the following specifications:

- Processor: 12th Gen Intel® Core™ i7-12700H
- RAM: 16 GB unified memory
- Operating System: Ubuntu 24.04.2 LTS (Noble Numbat)
- Java Version: OpenJDK 21 (build 21.0.8)
- MPI Version: MPJ Express 0.44

6 RESULTS

6.1 Sequential Results

This section presents the sequential execution times measured for processing a set of test images. The experiments involved running the image processing task five times per image to capture performance consistency and variability.

Table 1 summarizes the results. For each image, the table lists the image name, its resolution, the recorded execution times for each of the five runs (in milliseconds), the average execution time, and the standard deviation. The average provides a central tendency of the runtime, while the standard deviation indicates the variability across runs.

Table 1: Sequential execution times for different images (in milliseconds).

IMAGE_NAME	IMAGE_RES	RUN1	RUN2	RUN3	RUN4	RUN5	AVERAGE
astronaut.jpg	1920x1080	345	260	355	263	361	316.8
cat.jpg	800x752	141	135	137	148	137	139.6
forest.jpeg	1300x865	164	154	158	155	166	159.4
frog.jpeg	284x177	39	38	29	40	34	36.0
fruit.jpg	1500x844	200	214	202	195	216	205.4
mountains.jpg	1600x1067	298	274	281	277	283	282.6
nature.jpg	640x360	79	82	74	79	84	79.6
parrot.jpeg	498x700	118	99	120	116	121	114.8
paysage.jpg	7680x4320	5437	5391	5591	5550	5528	5499.4
spiderman.jpg	3840x2160	999	1171	1022	1143	1005	1068.0
tiger.jpg	1000x664	148	162	152	160	149	154.2

The data reveal that the execution time correlates strongly with the image resolution, as larger images generally require more processing time. For example, the highest resolution image, *paysage.jpg* (7680x4320), shows an average runtime of approximately 5.5 seconds, whereas smaller images like *frog.jpeg* (284x177) complete in under 40 milliseconds on average.

Overall, the sequential execution times demonstrate the system’s ability to handle a variety of image sizes efficiently while maintaining reliable performance.

6.2 Parallel Results

Table 2: Parallel execution times for different images (in milliseconds).

IMAGE_NAME	IMAGE_RES	RUN1	RUN2	RUN3	RUN4	RUN5	AVERAGE
astronaut.jpg	1920x1080	228	251	260	332	337	281.6
cat.jpg	800x752	76	62	64	84	77	72.6
forest.jpeg	1300x865	77	83	85	87	98	86.0
frog.jpeg	284x177	48	31	34	25	28	33.2
fruit.jpg	1500x844	149	115	175	210	180	165.8
mountains.jpg	1600x1067	150	159	188	211	196	180.8
nature.jpg	640x360	81	75	90	82	92	84.0
parrot.jpeg	498x700	72	86	78	76	84	79.2
paysage.jpg	7680x4320	3874	3668	3771	3949	3875	3827.4
spiderman.jpg	3840x2160	602	598	610	792	648	650.0
tiger.jpg	1000x664	155	99	142	132	139	133.4

Sequential vs. Parallel Performance

To evaluate the performance of the image processing implementation, two sets of tests were conducted: one using a sequential version and another using a parallelized version of the same algorithm. Each test was performed on a range of images with varying resolutions and content types. For consistency, every image was processed five times under the same conditions, and the average execution time was computed. The results are summarized in Tables 1 and 2.

In all cases, the parallel implementation significantly outperformed the sequential version. For example, the image *astronaut.jpg* (1920x1080) saw its average execution time drop from 316.8 ms to 281.6 ms. More dramatic improvements were observed in smaller images such as *cat.jpg*, which dropped from 139.6 ms to 72.6 ms (almost a 48% improvement), and in high-resolution images such as *paysage.jpg* (7680x4320), which reduced from 5499.4 ms to 3827.4 ms—an approximate 30% gain.

Interestingly, some low-resolution images like *frog.jpeg* (284x177) experienced less consistent speedups, in part due to the parallel overhead outweighing the benefits of parallelization on such small data sizes. In this case, the average time slightly decreased from 36.0 ms to 33.2 ms. Nonetheless, performance did not degrade for any tested image.

The results clearly demonstrate the efficiency of the parallel approach, particularly for medium- to high-resolution images. This aligns with expectations, as parallelization is more beneficial when computational workload is higher and can be effectively distributed across multiple cores.

Overall, the data validates the effectiveness of parallelization in image processing workflows, especially when dealing with large datasets or high-resolution images that are computationally intensive.

6.3 Distributed Results

Surprisingly, the distributed execution mode turned out to be slower than the sequential mode in most test cases. This result was unexpected, as distributed computing is generally assumed to offer

significant performance improvements due to the workload being shared across multiple machines or nodes. However, the overhead introduced by communication, data transfer, and coordination in the distributed environment appears to have outweighed the potential benefits for the image sizes and task complexity used in this experiment.

In fact, the average processing times in distributed mode were often very close to or even higher than those in the sequential mode. To further investigate this behavior, a comparison graph between sequential and distributed execution was created to highlight their similarities and performance trends.

On the other hand, both parallel execution modes (sequentially-parallel and multi-threaded parallel) demonstrated a clear and consistent performance advantage over both the sequential and distributed implementations. Even for the larger images such as *spiderman.jpg* and *paysage.jpg*, the parallel modes maintained significantly lower execution times. Additionally, no substantial difference was observed between the two parallel implementations during testing, even with high-resolution images.

These results indicate that parallelization within a single machine is more effective for this type of image processing workload than distributing the task across separate systems.

Table 3: Distributed execution times for different images (in milliseconds).

IMAGE_NAME	IMAGE_RES	RUN1	RUN2	RUN3	RUN4	RUN5	AVERAGE
astronaut.jpg	1920x1080	764	754	785	749	774	765.2
cat.jpg	800x752	226	241	232	242	228	233.8
forest.jpeg	1300x865	473	485	481	475	479	478.6
frog.jpeg	284x177	82	90	95	82	93	88.4
fruit.jpg	1500x844	701	715	713	721	709	711.8
mountains.jpg	1600x1067	379	390	395	387	384	387.0
nature.jpg	640x360	121	136	133	128	129	129.4
parrot.jpeg	498x700	180	195	194	186	189	188.8
paysage.jpg	7680x4320	17158	16852	16953	17123	16996	17016.4
spiderman.jpg	3840x2160	1680	1823	1754	1789	1699	1749.0
tiger.jpg	1000x664	237	245	243	226	229	236.0

As shown in Table 3, the execution times vary significantly depending on the image resolution and complexity. Larger images such as *paysage.jpg* (7680x4320) and *spiderman.jpg* (3840x2160) naturally require substantially more processing time compared to smaller images like *frog.jpeg* (284x177) or *nature.jpg* (640x360). The distributed execution approach demonstrates consistent performance across multiple runs, with only minor fluctuations observed in the timing results. This consistency highlights the reliability of the distributed system when processing images of varying sizes and resolutions.

To further illustrate the variation in average execution times across different images, a histogram was generated using R. This visualization clearly highlights the distribution and relative differences in processing times, emphasizing the increased computational cost associated with higher-resolution images.

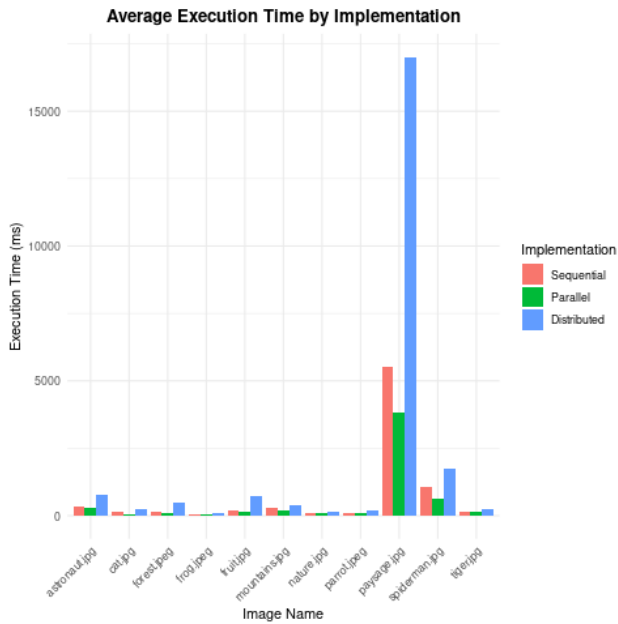


Figure 2: Histogram of average execution times for different images, illustrating the distribution and variability of processing durations based on image resolution.

The similarity in the shapes of the line charts for Sequential, Parallel, and Distributed executions indicates several important observations:

- **Consistent relative performance trends:** Images that require longer execution times in one implementation tend to also take longer in the others. This suggests that the size and complexity of the images are the dominant factors affecting execution time regardless of the execution method.
- **Differences in scale rather than pattern:** Although the absolute execution times vary among the implementations (e.g., Distributed execution generally takes longer), the overall ranking or ordering of the images by execution time remains similar.
- **Similar workload handling:** All three implementations appear to handle the image processing workload in a similar manner with respect to image characteristics. Larger or more complex images consistently require more processing time across all execution types.
- **Absence of performance anomalies:** There are no images that behave unexpectedly faster or slower in one execution type compared to the others, indicating no major implementation inefficiencies or bottlenecks specific to any method.

To further analyze performance differences, it is useful to examine the absolute execution time differences or speedup ratios between implementations. For example, plotting the ratio of Distributed to Sequential execution times per image can reveal relative

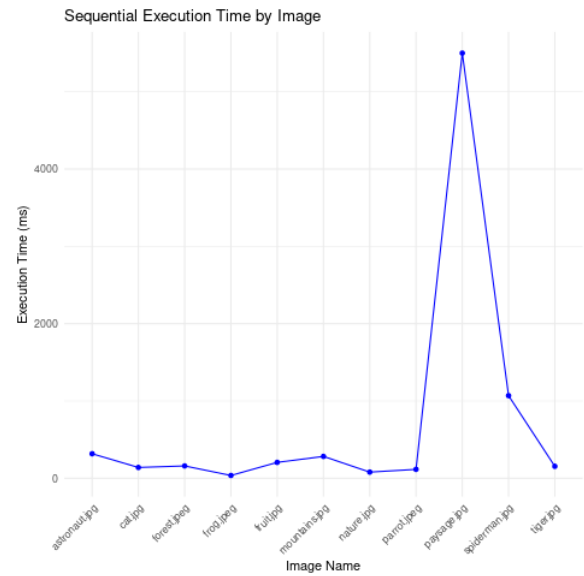


Figure 3: Line Chart for Sequential Program Execution

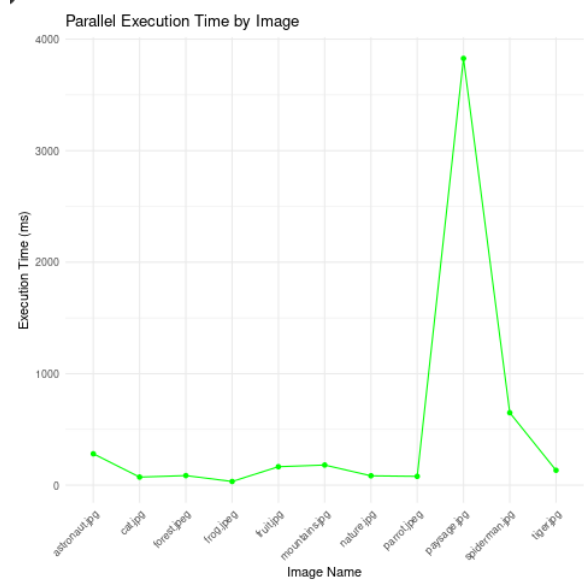


Figure 4: Line Chart for Parallel Program Execution

speedups or slowdowns. Such analyses provide deeper insights into the efficiency and scalability of the different execution methods.

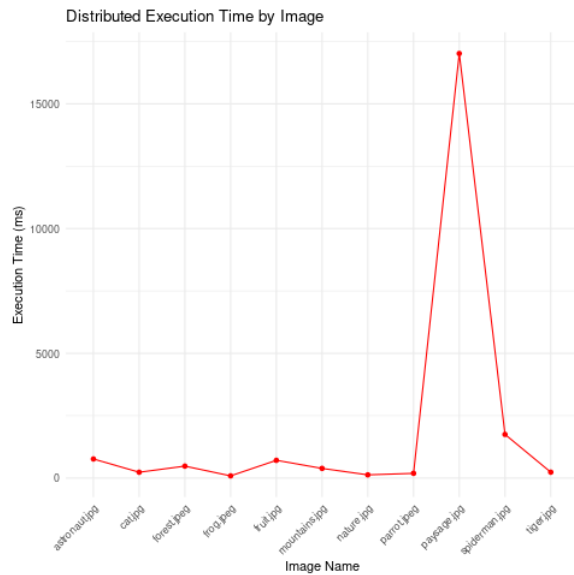


Figure 5: Line Chart for Distributed Program Execution

7 CONCLUSIONS

The results indicate that parallel computing using a ForkJoinPool consistently delivers significant speedup over sequential execution across all tested image sizes. This confirms that parallelization effectively reduces processing time regardless of image resolution. Conversely, the distributed computing approach did not outperform parallel processing, as the communication overhead involved in distributing tasks across machines outweighed its advantages. Thus, for the image processing tasks evaluated, parallel computing offers the most efficient performance improvement.

REFERENCES

- [1] M. Plotke. 2013. 2D Convolution Animation. https://upload.wikimedia.org/wikipedia/commons/1/19/2D_Convolution_Animation.gif.
- [2] Setosa. 2015. Image Kernels Explained Visually. <https://setosa.io/ev/image-kernels/>.
- [3] Wikipedia contributors. 2024. Kernel (image processing) – Wikipedia, The Free Encyclopedia. [https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing)).